

# Clicktick

## Real-time Concurrency Analytics at Streaming Scale

Vikash Kumar · Muni Siva | Click-a-thon 2026

# The Core Challenge

## Problem:

Count truly active viewers (foreground only) at scale—excluding paused, backgrounded, and idle sessions—while staying sub-second for dashboards and absorbing continuous heartbeat updates.

## Our Approach:

- ✓ Normalized active intervals: Never modify a closed interval
- ✓ Per-minute concurrency deltas: Pre-aggregated +1/-1 events for fast peak/avg
- ✓ Lightweight open-session tracking: New heartbeats update only last\_seen\_ts
- ✓ ClickHouse materialized views: Incremental aggregation, partition pruning, dimension-aware filtering

## Architecture: Ingestion → Aggregation → Serving

### Raw Events

- Heartbeats
- Play/Pause
- BG/FG

### Normalize

- Active Intervals
- Deltas (+1/-1)
- Open Sessions

### Pre-aggregate

- Minute Concurrency
- Peak/Avg
- Dimensions

### Query

- Grafana
- LibreChat
- Sub-second

# Why This Design Wins



## Speed

Serving tables are pre-aggregated minute deltas. Queries sum only relevant partitions (platform, date, content). No session history scan = sub-second dashboards.



## Accuracy

Heartbeat gaps, pauses, backgrounding all mapped to interval end-times. Only active ranges counted. Foreground-only by design.



## Incremental

Open sessions updated by `last_seen_ts` only. Closed intervals never reshaped. New heartbeats incrementally flow through MVs, no rebuild.

# Data Model: Three Simple Tables

**session\_active\_intervals\_clean**

*Source of truth*

⟨video\_session\_id, interval\_start, interval\_end, platform, country, content\_id⟩

**concurrency\_deltas**

*Pre-aggregated serving*

⟨minute\_bucket, +1/-1 delta, platform, country, content\_id, video\_type⟩

**open\_session\_intervals**

*Live sessions*

⟨session\_id, interval\_start, last\_seen\_ts, heartbeat\_timeout⟩

# Filter-Fast: Dimensions Embedded in Serving Layer

*Every filter dimension is a sort key + partition strategy in concurrency\_deltas*

Platform

JIO\_ANDROID\_TV, ANDROID\_PHONE, IPHONE, ...

*Partition key*

Country

IN, US, UK, ...

*Sort key cluster*

Content ID

Numeric, zero hash collisions

*Index for slicing*

Time Grain

Minute partitions by date

*Date-based pruning*

**Result: Any filter combination queries only relevant blocks. No full table scan.**

## Validated Results

### Concurrency Accuracy

✓ Foreground-only, excludes paused/BG/idle

### Peak Concurrency

✓ Max over minute/hour/day grains

### Average Concurrency

✓ Arithmetic mean validated

### Query Latency

✓ Sub-second on minute-grain with filters

### Live Session Handling

✓ Incremental heartbeat absorption

# Conversational Interface: Natural Language on Concurrency

## LibreChat + ClickHouse MCP Server

💬 "What was peak concurrency on Android during the live event?"

💬 "Show average concurrency by country for India vs Pakistan."

💬 "Which platform had the most viewers in the last hour?"

💬 "Alert me if concurrency drops >20% in the next 30 minutes."

# Live Demo



## Grafana Dashboard

- ✓ Real-time active users
- ✓ Peak concurrency/platform
- ✓ Minute-grain trends
- ✓ Dimension filtering



## LibreChat + MCP

- ✓ Natural language queries
- ✓ ClickHouse SQL generation
- ✓ Sub-second responses
- ✓ No schema needed

Dashboard - <https://scarletcorn757.grafana.net/public-dashboards/ec4dd23ed92948348c9586e491226971>

Librechat with clicktick bot - <https://guiding-monster-publicly.ngrok-free.app/c/new?endpoint=ollama&model=gpt-oss&projectId=6a6e2bdbed5b4a34774a88cf>

Email: [kvvik2020@gmail.com](mailto:kvvik2020@gmail.com) | Password: clicktick

# Accurate • Fast • Scalable

Foreground-only streaming concurrency at petabyte scale

**Team Clicktick**

Vikash Kumar · Muni Siva